



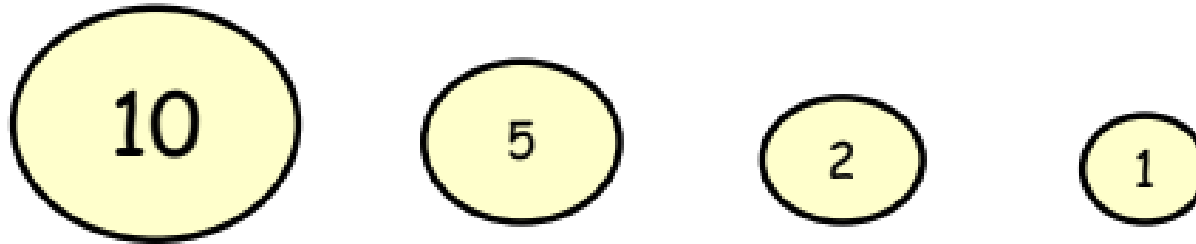
Department of Computer and Information Science

อัลกอริทึมเชิงละโมภ (Greedy algorithm) Part 1

อ.ดร.ลือพล พิพานเมฆาภรณ์
luepol.p@sci.kmutnb.ac.th

ปัญหาแลกเหรียญ (Coin changing problem)

- มีเหรียญ 1 บาท 2 บาท 5 บาท และ 10 บาท จำนวนไม่จำกัด ต้องการทอนเงิน 57 บาท จะต้องใช้เหรียญเหล่านี้อย่างน้อยที่สุดกี่เหรียญ

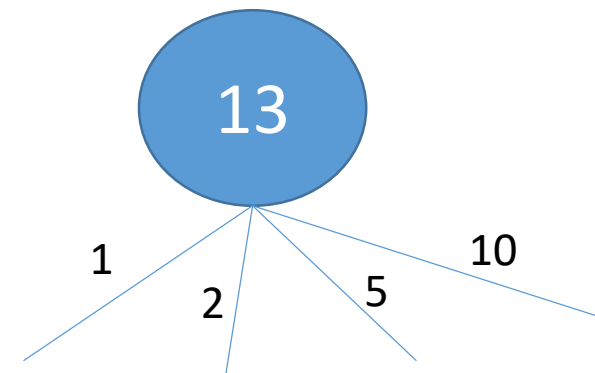


Greedy algorithm

- ในการหาคำตอบที่ดีที่สุด (optimal solution) ภายใต้เงื่อนไขบังคับ (constraint) ด้วยวิธี brute force อาจต้องการเวลาและทรัพยากรค่อนข้างสูง อัลกอริทึม greedy ถูกออกแบบมาเพื่อลดเวลาในการหาคำตอบ ภายในเวลาที่เหมาะสม
- แนวคิดของอัลกอริทึมเชิงละโมภจะเลือกตัวเลือกที่ดีที่สุดในแต่ละขั้นตอน (local optimal choice) โดยใช้เกณฑ์ในการเลือก (greedy criterion) และคาดหวังว่าการเลือกแบบนี้จะนำไปสู่ผลลัพธ์รวมที่ดีที่สุด (global optimal solution)

หลักการของอัลกอริทึมเชิงละโมภ

- ในการเลือกตัวเลือกที่ดีที่สุด จะอาศัยเกณฑ์เชิงละโมภ (greedy criterion) เพื่อประเมินคุณภาพของคำตอบในขณะนั้น
- ตัวอย่างเช่น เกณฑ์เชิงละโมภอาจถูกกำหนดเป็น
แลกเหรียญที่มีมูลค่ามากที่สุดซึ่งสามารถแลกเงินได้ในขณะนั้นก่อน



$$\text{Optimal Solution} = \text{Greedy Choice} + \text{Subproblem Solution}$$

Exercise

- จงเขียนฟังก์ชัน greedy เพื่อทอนเหรียญให้มีจำนวนน้อยที่สุด กำหนดให้ อารเรย์ของมูลค่าเหรียญ $D[] = \{1, 5, 2, 10\}$, $n = 4$ และ $W = 13$

```
int coin_change(int W, int D[], int n)
{
    int cnts = 0;

    .....

    return cnts;
}
```

การตรวจสอบความถูกต้องของ Greedy

- ก่อนนำ greedy algorithm ไปใช้งาน ควรต้องมีการพิสูจน์ความถูกต้องของ greedy ว่าให้ผลลัพธ์ถูกต้องหรือไม่ โดยใช้หลักการพิสูจน์โดยอุปนัย (induction proof)

กรณีพื้นฐาน เมื่อ $N = K[i]$ และ $K = \{1, 2, 5, 10\}$ พิสูจน์ว่า greedy ใช้เหรียญน้อยที่สุด

$N = 1$ แลกเหรียญมีมูลค่า 1 บาท ใช้น้อยที่สุด 1 เหรียญ

$N = 2$ แลกเหรียญมีมูลค่า 2 บาท ใช้น้อยที่สุด 1 เหรียญ

$N = 5$ แลกเหรียญมีมูลค่า 5 บาท ใช้น้อยที่สุด 1 เหรียญ

$N = 10$ แลกเหรียญมีมูลค่า 10 บาท ใช้น้อยที่สุด 1 เหรียญ

การตรวจสอบความถูกต้องของ Greedy

กรณีทั่วไป สมมติ greedy algorithm จะให้ผลลัพธ์ที่ถูกต้องสำหรับทุกจำนวนเงิน $0 \leq k \leq N$ พิสูจน์สำหรับ $N+1$ ว่า greedy algorithm ให้ผลลัพธ์ที่ถูกต้องสำหรับจำนวนเงิน $N+1$

สมมติให้ $N+1$ คือ เงิน 3 บาท และเหรียญมีมูลค่า 10, 5, 2, 1 ดังนั้น greedy algorithm จะแลกเหรียญที่มีมูลค่ามากที่สุดที่น้อยกว่าหรือเท่ากับ 3 บาท ซึ่งก็คือ เหรียญมูลค่า 2 บาท

$3 - 2 = 1$ บาท ใช้ 1 เหรียญ

$1 - 1 = 0$ บาท ใช้ 1 เหรียญ กรณีพื้นฐานเรารับประกันว่าใช้เหรียญน้อยที่สุด

ดังนั้นรวมเป็น 2 เหรียญ

การตรวจสอบความถูกต้องของ Greedy

- อย่างไรก็ตาม หากเปลี่ยนมูลค่าเหรียญเป็น $\{1, 3, 4\}$ greedy algorithm อาจไม่รับประกันผลลัพธ์

เมื่อ $N = 6$

greedy จะแลกเหรียญ 4 บาท $6 - 4 = 2$ บาท 1 เหรียญ

greedy จะแลกเหรียญ 1 บาท $2 - 1 = 1$ บาท 1 เหรียญ

greedy จะแลกเหรียญ 1 บาท $1 - 1 = 0$ บาท 1 เหรียญ

รวม 3 เหรียญ แต่ optimal solution จะเป็นเหรียญ 3 บาท 2 เหรียญ คือ $3 + 3 = 6$

Greedy Algorithm

- ข้อดี:

- ง่ายในการพัฒนา
- Time Complexity ไม่สูงมาก $O(n^k)$ (polynomial)

- ข้อเสีย:

- ไม่รับประกันผลลัพธ์ optimal solution แต่จะได้คำตอบที่มีความใกล้เคียง (approximate solution)

ปัญหา 0/1 knapsack

- คำตอบของปัญหา 0/1 Knapsack สามารถแสดงในรูปของไบนารีเซตโดยที่ไอเท็มซึ่งถูกเลือก ('1') และไม่ถูกเลือก ('0')

Object	Weight	Profit
o1	18	25
o2	15	24
o3	10	15
o4	5	8

X	Profit	Weight
{0, 0, 1, 1}	$15 + 8 = 23$	$10 + 5 = 15$

Greedy criteria for 0/1 knapsack

สมมติ $N = 4$ และน้ำหนักถุงที่รับได้คือ $M = 25$ กำหนดให้ของแต่ละชิ้นมีมูลค่า

$(p_1, p_2, p_3, p_4) = (25, 24, 15, 8)$ และน้ำหนักคือ $(w_1, w_2, w_3, w_4) = (18, 15, 10, 5)$

1. เลือกแบบสุ่ม (Random Selection)
2. เลือกของน้ำหนักน้อยที่สุดใส่ก่อน (Minimize weight)
3. เลือกของที่มีมูลค่ามากที่สุดใส่ก่อน (Maximize Profit)

Greedy solutions

Object	Weight	Profit
o1	18	25
o2	15	24
o3	10	15
o4	5	8

$$M = 25$$

1. เรียงลำดับสิ่งของตามน้ำหนักที่น้อยที่สุดก่อน o4->o3->o2->o1

$$X = \{0, 0, 1, 1\} \quad P = 15 + 8 = 23$$

2. เรียงลำดับสิ่งของตามมูลค่าจากมากไปน้อย o1->o2->o3->o4

$$X = \{1, 0, 0, 0\} \quad P = 25$$

Density-based greedy criterion

- เลือกสิ่งของโดยนำของซึ่งมีมูลค่ามากแต่น้ำหนักน้อยใส่ถุงก่อน
- กำหนดให้ความหนาแน่น (density) คือสัดส่วนของมูลค่าและน้ำหนักของวัตถุ ตัวอย่างเช่น วัตถุ A มีมูลค่า 25 น้ำหนัก 18 จะมีความหนาแน่นคือ $25/18 = 1.39$
- หลักการของ Greedy
 - เรียงลำดับของแต่ละชิ้นตามความหนาแน่นของมันจากน้อยไปมาก
 - สำหรับแต่ละขั้นตอน เลือกของที่มีความหนาแน่นมากใส่ลงไปก่อน จนกระทั่งไม่สามารถใส่ได้ (ถุงขาด)

ตัวอย่าง

สมมติ $n = 4$ และน้ำหนักถุงที่รับได้คือ $M=25$ กำหนดให้ของแต่ละชั้นมีมูลค่า

$(p_1, p_2, p_3, p_4) = (25, 24, 15, 8)$ และน้ำหนักคือ $(w_1, w_2, w_3, w_4) = (18, 15, 10, 5)$

o_1	o_2	o_3	o_4
$25/18=1.39$	$24/15=1.6$	$15/10=1.5$	$8/5=1.6$

ใส่ของลงไปในถุงตามลำดับ : $o_2 \rightarrow o_4 \rightarrow o_3 \rightarrow o_1$

(x_1, x_2, x_3, x_4)
0 1 0 1

profit
 $24+8 = 32$

weight
 $15+5 = 20$

Fractional knapsack

- ปัญหา 0/1 Knapsack อนุญาตให้ใส่ของลงไปในถุงทั้งวัตถุ ซึ่งมักจะทำให้ถุงเหลือน้ำหนักบางส่วนทิ้งไว้ ตัวอย่างเช่น ในตัวอย่างที่แล้วเราจะเหลือน้ำหนัก 5 หน่วยที่ไม่สามารถใส่ของได้แล้ว
- ปัญหา Fractional knapsack จะเพิ่มเงื่อนไขของปัญหาคือว่าเราสามารถแบ่งของเพื่อนำไปใส่ในถุงได้ แต่มูลค่าของสิ่งนั้นจะลดลงตามสัดส่วนที่แบ่งด้วยเช่นกัน

	o1	o2	o3	o4
P	25	24	15	8
W	18	15	10	5
(x1, x2, x3, x4)				
1/4 0 1/2 1				
profit				
(25*1/4)+ (15*1/2)+ 8 = 21.75				
weight				
(15*1/4)+(5*1/2)+5 = 11.25				

Exercise: fractional knapsack

- วิธีการเลือกตามความหนาแน่นของวัตถุที่ใช้ในปัญหา 0/1 Knapsack จะยังให้คำตอบที่เป็น Optimal หรือไม่?

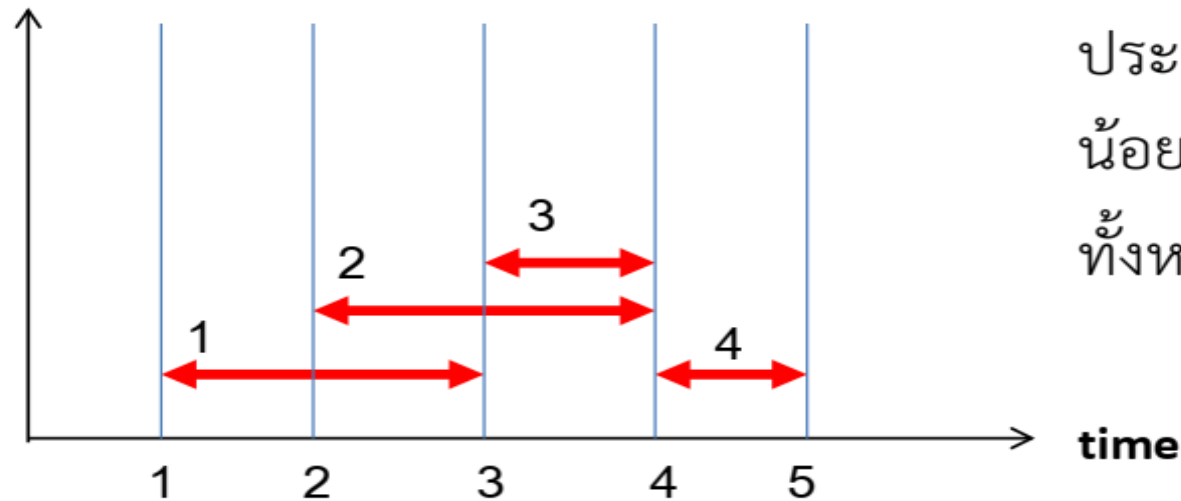
o1	o2	o3	o4
$25/18=1.39$	$24/15=1.6$	$15/10=1.5$	$8/5=1.6$

ใส่ของลงไปในถุงตามลำดับ : o2 $\rightarrow$ o4 $\rightarrow$ o3 $\rightarrow$ o1

(x1, x2, x3, x4)	profit	weight
0 1 $1/2$ 1	$24+8+(15*1/2) = 39.5$	$15+5+(10*1/2) = 25$

Activity Selection problem

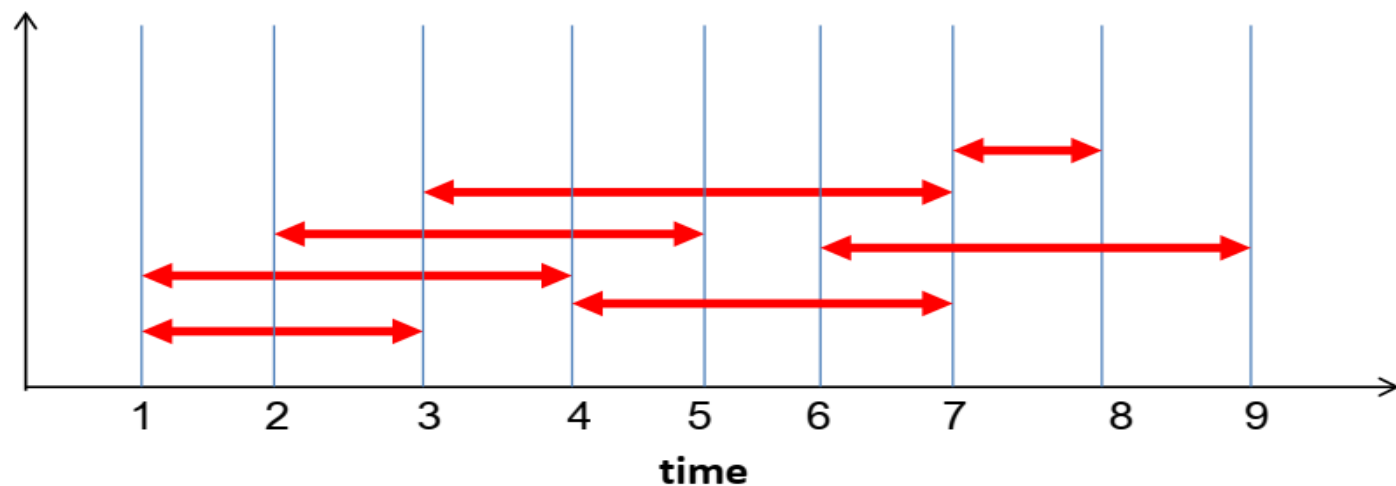
- กำหนดมีเซตของงานทั้งสิ้น n งาน ซึ่งรอที่จะถูกประมวลผลโดย machine แต่ละงานจะประกอบไปด้วย (s_i, f_i) ซึ่งหมายถึงเวลาเริ่มต้นและเวลาสิ้นสุดของการประมวลผลงานที่ i เนื่องจาก machine สามารถประมวลผลได้เพียงครั้งละ 1 งาน ดังนั้นหาก 2 งานมีเวลาซ้อนทับกัน (overlap time) machine จะต้องเลือก 1 งานมาประมวลผล และจะต้องยกเลิกอีกงานไป



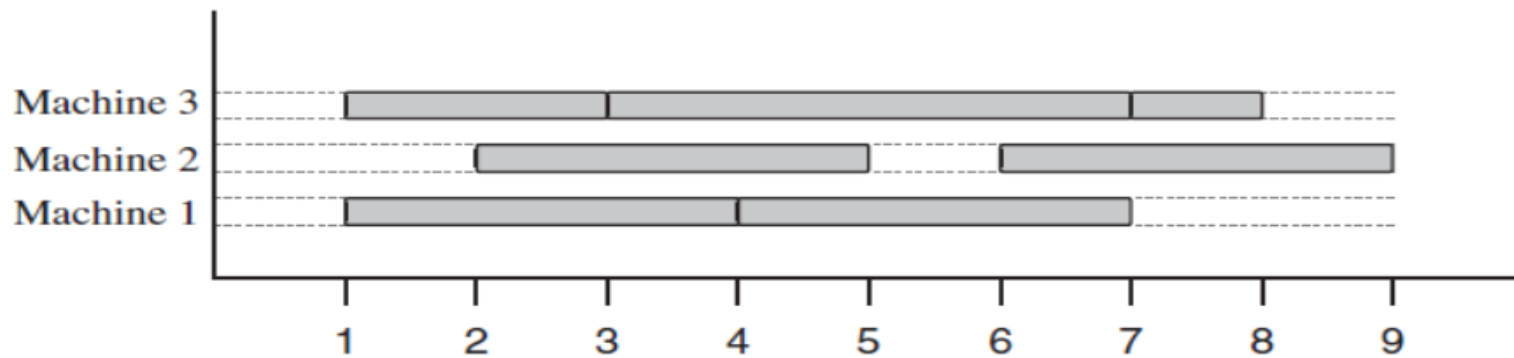
ประเด็นคือต้องการหาจำนวน machine ที่น้อยที่สุดเพื่อรองรับการประมวลผลงานทั้งหมด

หากเลือกประมวลผลงาน 2 จะต้องยกเลิกงาน 1 และงาน 3 หรือต้องการ machine เพิ่มอีก 1 ตัว เพื่อประมวลผลงาน 1 และงาน 3 ตามลำดับ

ตัวอย่าง



จะต้องใช้ machine อย่างน้อยที่สุด 3 ตัวในการประมวลผลงานทั้งหมด โดยไม่ขัดแย้ง



Greedy strategies

Longest first strategy:

เลือกงานที่มีระยะเวลาระหว่าง start time และ end time ที่ประมวผลนานที่สุดก่อน

Smallest interval strategy:

เลือกงานที่มีระยะเวลาระหว่าง start time และ end time ที่ประมวผลน้อยที่สุดก่อน

Least conflict first strategy:

เลือกงานที่มีช่วงเวลาประมวผลคาบเกี่ยวกับงานอื่นน้อยที่สุดก่อน

Earliest end-time strategy:

เลือกงานที่มีเวลาสิ้นสุดเร็วที่สุดก่อน

Activity	Start Time	End Time
A	1	4
B	3	5
C	0	6
D	5	7
E	3	8
F	5	9
G	6	10
H	8	11

Greedy algorithm for activity selection

Algorithm TaskSchedule(T):

Input: A set T of tasks, such that each task has a start time s_i and a finish time f_i

Output: A nonconflicting schedule of the tasks in T using a minimum number of machines

$m \leftarrow 0$ // optimal number of machines

while $T \neq \emptyset$ **do**

 remove from T the task i with smallest start time s_i

if there is a machine j with no task conflicting with task i **then**

 schedule task i on machine j

else

$m \leftarrow m + 1$ // add a new machine

 schedule task i on machine m

แบบฝึกหัด

กำหนดให้ greedy criterion เลือกโปรแกรมที่เวลาเริ่มเร็วที่สุดก่อน (earliest start time) จงแสดงวิธีการ greedy เพื่อค้นหาจำนวน CPU ที่น้อยที่สุดที่จะต้องใช้ในการประมวลผลโปรแกรมทั้งหมด

โปรแกรม	1	2	3	4	5	6	7	8	9	10
เวลาเริ่ม	1	3	0	5	3	5	6	8	8	2
เวลาสิ้นสุด	4	5	6	7	8	9	10	11	12	13

การบีบอัดข้อมูล (Data compression)

ปกติแล้วคอมพิวเตอร์จัดเก็บไฟล์ในลักษณะที่เป็น **fixed-length binary code** ตัวอย่างเช่นสมมติไฟล์หนึ่งประกอบไปด้วยตัวอักษรและความถี่ที่ใช้งานดังนี้

character	Code	frequency
a	0000 0001	10
e	0000 0010	15
i	0000 0101	12
s	1000 0000	3
t	0101 1100	4
r	0100 1100	1

TOTAL $45 * 8 = 360$ bits

การบีบอัดข้อมูล (Data compression)

จะเห็นได้ว่าค่อนข้างจะสิ้นเปลืองจำนวนบิตในการจัดเก็บมาก เนื่องจากมีตัวอักษรที่แตกต่างกันในไฟล์เพียง 6 ตัวเท่านั้น จึงเป็นไปได้ว่า เราสามารถลดจำนวนบิตลงให้สัมพันธ์กับความแตกต่างของตัวอักษรในไฟล์ได้

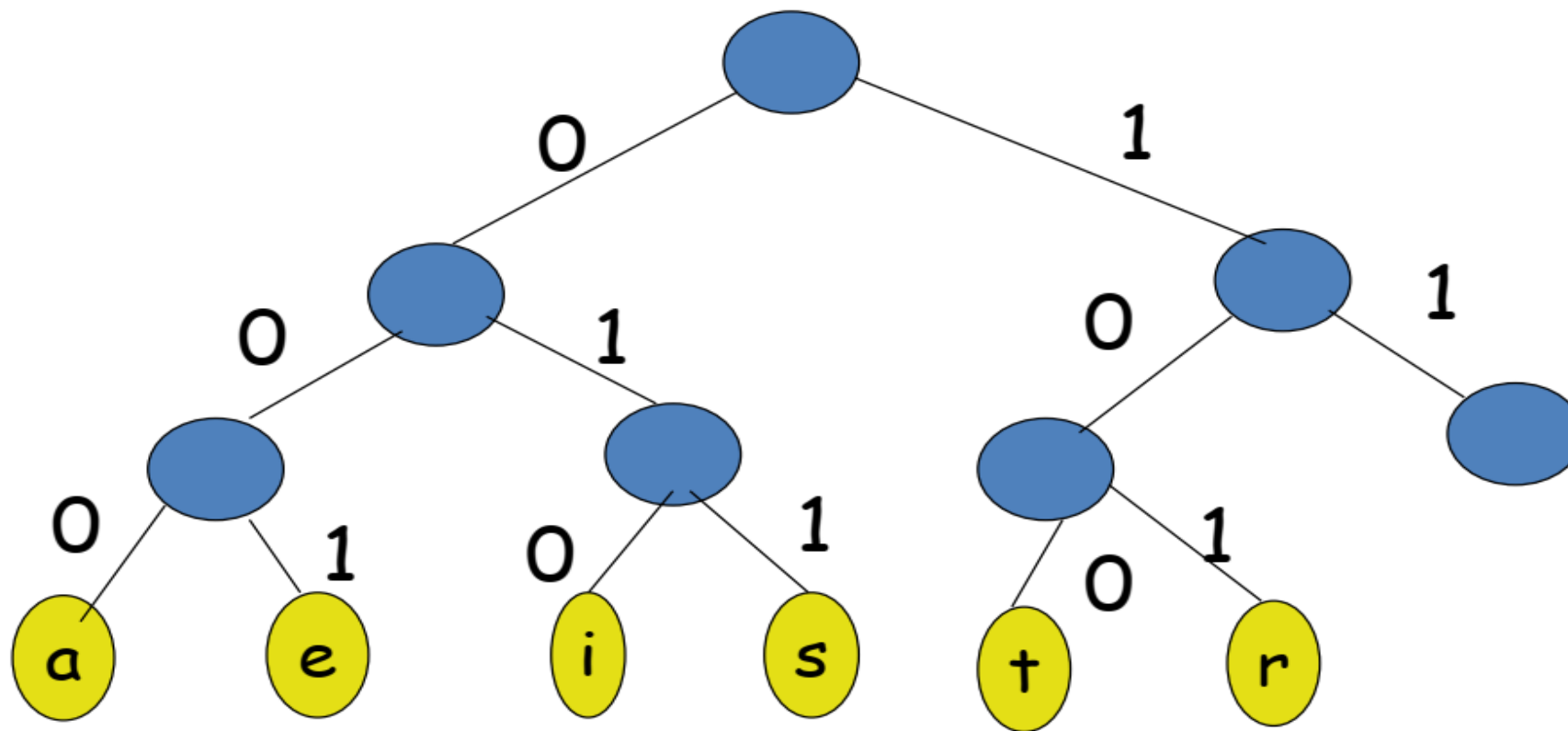
$$\#n \leq 2^{bit}$$

character	Code	frequency
a	000	10
e	001	15
i	010	12
s	011	3
t	100	4
r	101	1

TOTAL 45 * 3 = 135 bits

Binary encoding

เราสามารถแสดงต้นไม้ของรหัสไบนารี เพื่อความรวดเร็วสำหรับการถอดรหัสดังนี้

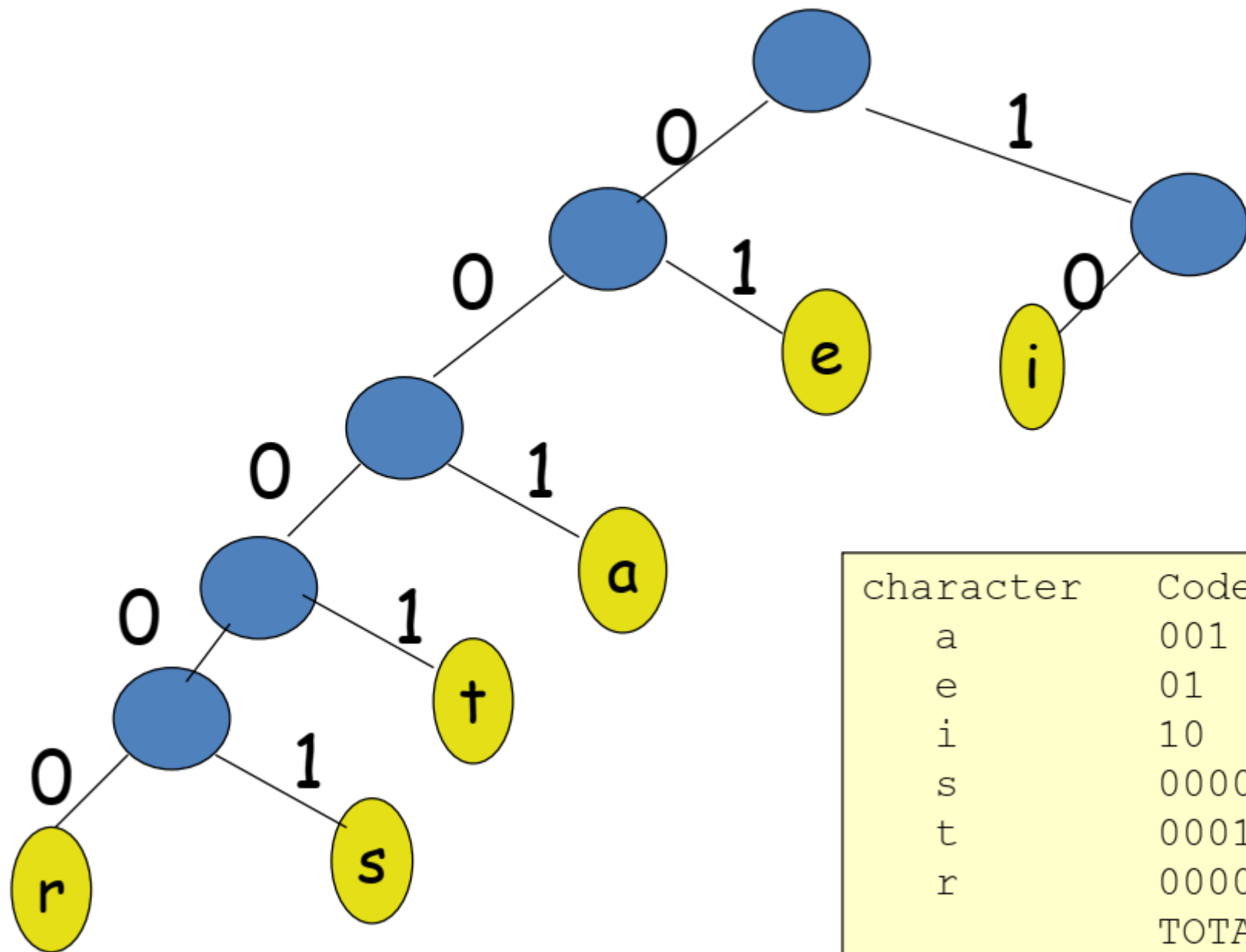


Variable-length binary coding

อย่างไรก็ตามถึงแม้ว่าจะลดพื้นที่ในการจัดเก็บได้ แต่ก็ยังคงไม่ optimal อยู่ดี ลองพิจารณาตัวอย่างด้านล่าง

character	Code	frequency
a	000	10
e	11	15
i	010	12
s	011	3
t	100	4
r	101	1

$$\text{TOTAL} \quad (30 * 3) + (15 * 2) = 90 + 30 = 120 \text{ bits}$$



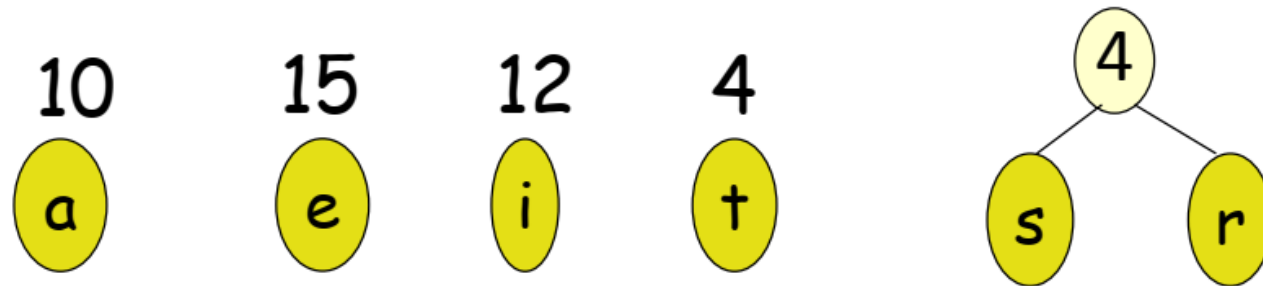
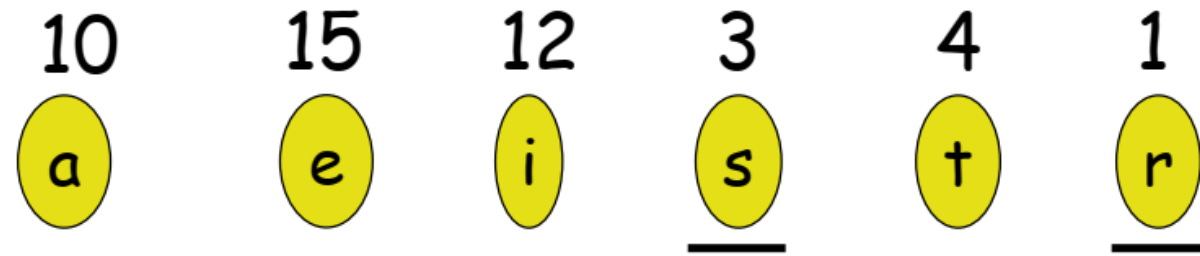
character	Code	frequency
a	001	10
e	01	15
i	10	12
s	00001	3
t	0001	4
r	00000	1
TOTAL		= 112 bits

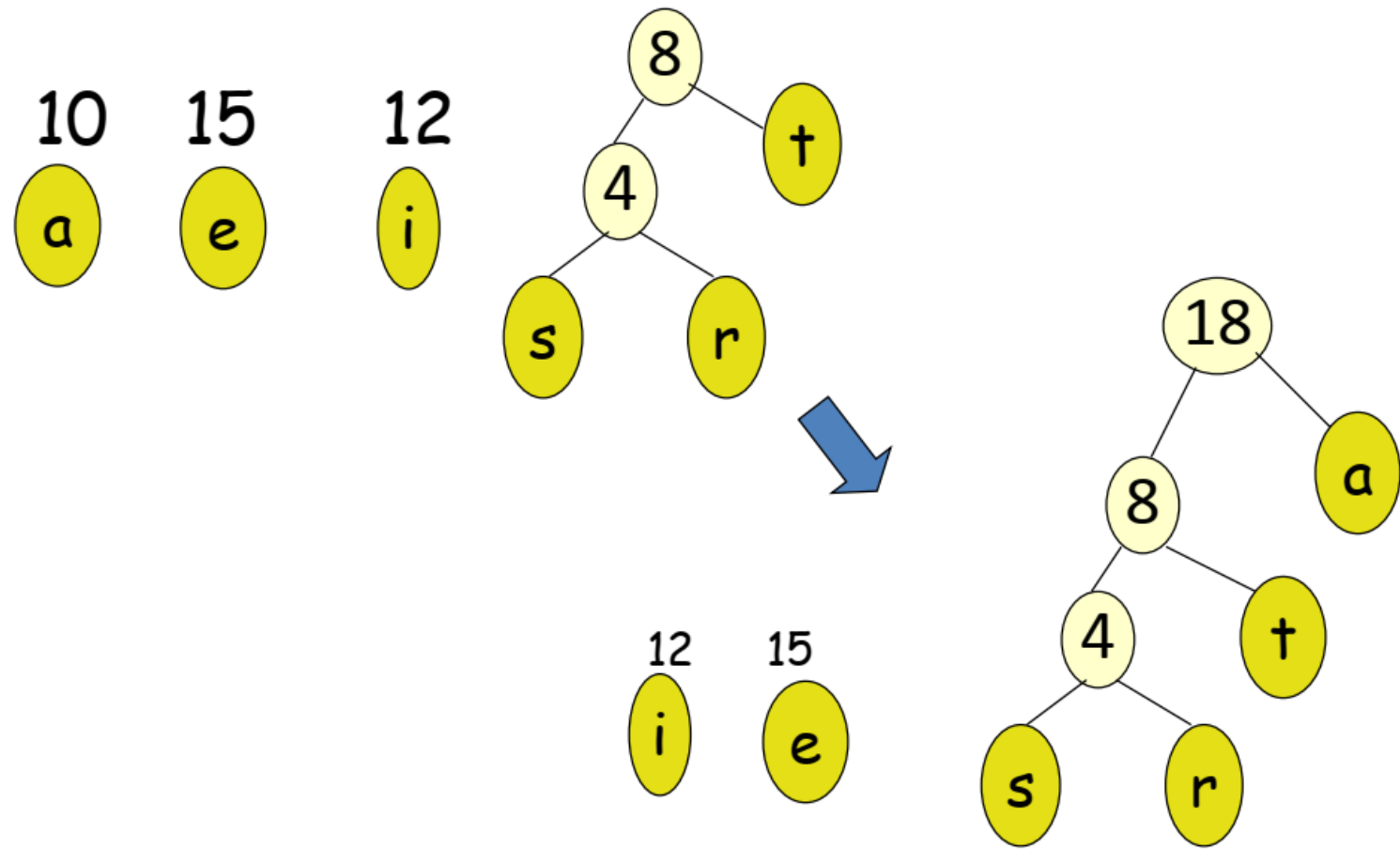
Huffman coding

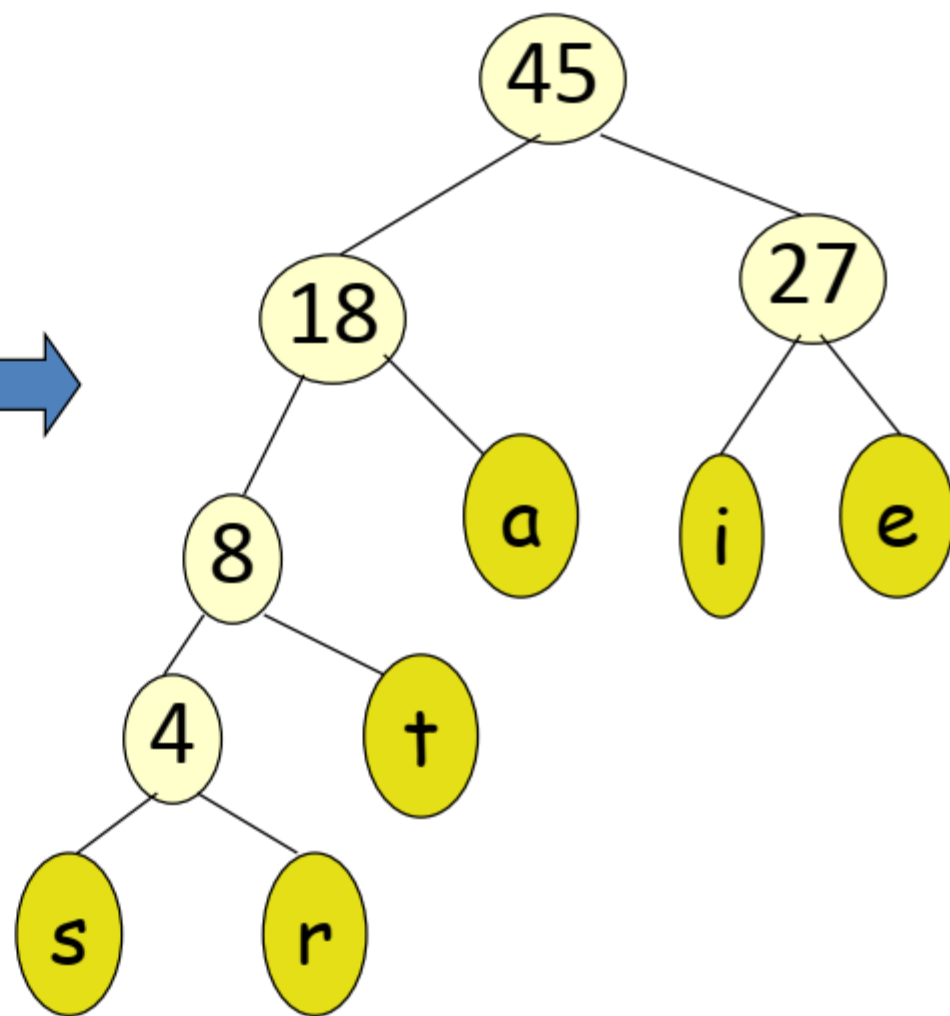
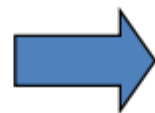
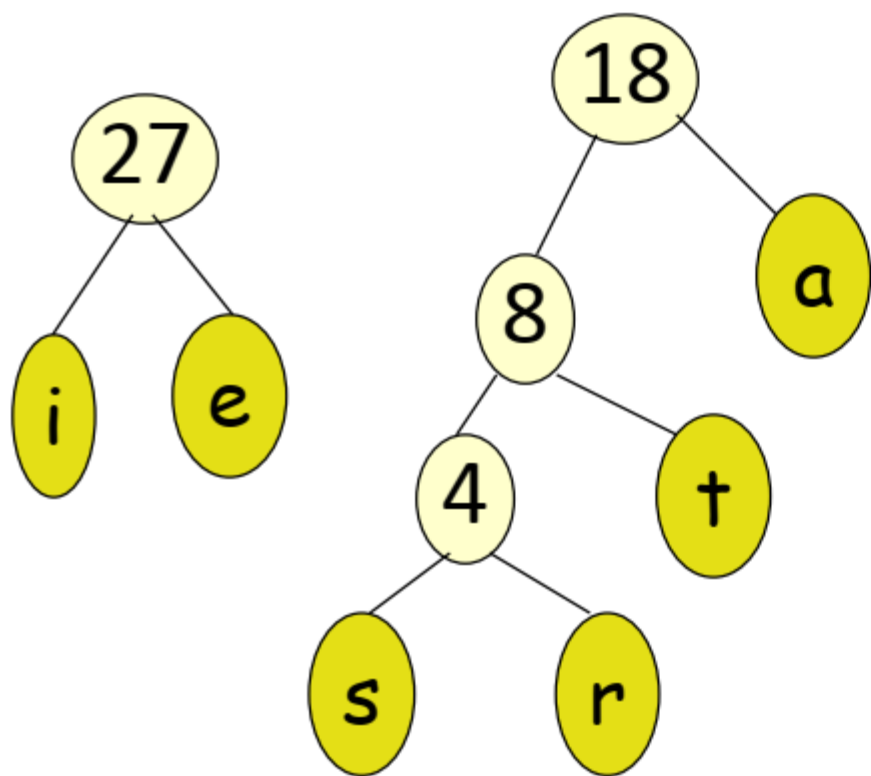
Letter ↕	Relative frequency in the English language ↕	
a	8.167%	
b	1.492%	
c	2.782%	
d	4.253%	
e	12.702%	
f	2.228%	
g	2.015%	
h	6.094%	
i	6.966%	
j	0.153%	
k	0.772%	
l	4.025%	
m	2.406%	
n	6.749%	
o	7.507%	
p	1.929%	
q	0.095%	
r	5.987%	
s	6.327%	
t	9.056%	
u	2.758%	
v	0.978%	
w	2.360%	
x	0.150%	
y	1.974%	
z	0.074%	

1. เริ่มจากนำทุกตัวอักษรและความถี่ หรือความน่าจะเป็น (probability) ก็ได้ ของมันมาสร้างเป็นต้นไม้ย่อย
2. หยิบต้นไม้ย่อย 2 ต้นที่มีความถี่ต่ำสุดมาเชื่อมกัน โดยสร้าง node ใหม่เป็นโหนดเชื่อมแล้วนำความถี่ของต้นไม้ย่อยดังกล่าวมารวมกัน
3. ทำเช่นนี้ไปเรื่อยๆ จนกระทั่งต้นไม้ย่อยทั้งหมดรวมเป็นต้นไม้ต้นเดียว
4. กำหนดรหัสให้กับแต่ละตัวอักษร

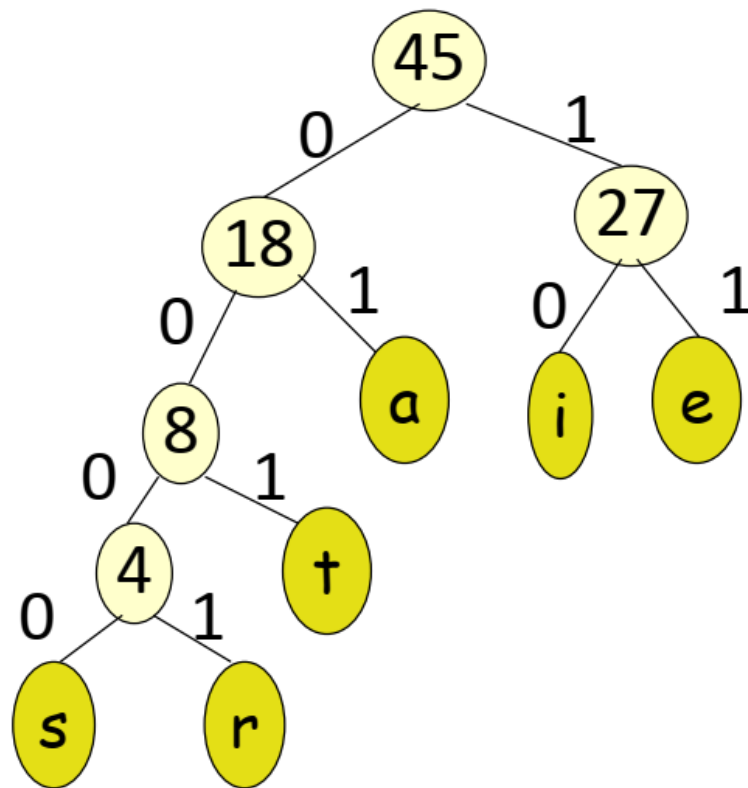
Huffman coding







ให้กิ่งซ้ายมีค่าเป็น 0 และกิ่งขวามีค่าเป็น 1 จากนั้นรหัสของแต่ละตัวอักษรจะเริ่มไล่จาก root node



character	Code	frequency
a	01	10
e	11	15
i	10	12
s	0000	3
t	001	4
r	0001	1

$$\begin{aligned}\text{TOTAL} &= \\ & (10 \times 2) + (2 \times 15) + (2 \times 12) + (4 \times 3) + (3 \times 4) + (4 \times 1) \\ &= 20 + 30 + 24 + 12 + 12 + 4 \\ &= 102 \text{ bits}\end{aligned}$$

แบบฝึกหัด

1. สร้าง Huffman code จากชุดตัวอักษรและความถี่ที่กำหนดให้
2. เข้ารหัสข้อความ "**WXYZ**" ด้วยรหัส Huffman ที่ได้
3. ถอดรหัส "0101011011000" โดยใช้ตารางรหัส Huffman

ตัวอักษร	ความถี่
X	5
Y	9
W	12
Z	13
V	16
U	45